

El Antipatrón del delay(): Control de Múltiples LEDs

Reporte de práctica — Sistemas Programables

1. Descripción del proyecto

Este proyecto tiene como objetivo evidenciar, de forma práctica, el problema que representa el uso de la función `delay()` cuando se necesita controlar varios eventos con distintos tiempos de forma simultánea. Se plantea un circuito con tres LEDs, cada uno con una intención de parpadeo diferente: LED1 cada 500 ms, LED2 cada 1000 ms y LED3 cada 1500 ms.

A primera vista, el código parece cumplir con lo solicitado, ya que cada LED tiene su propio bloque con su propio tiempo de encendido y apagado. Sin embargo, la forma en que está estructurado el programa provoca un comportamiento muy distinto al esperado.

2. Objetivos de aprendizaje

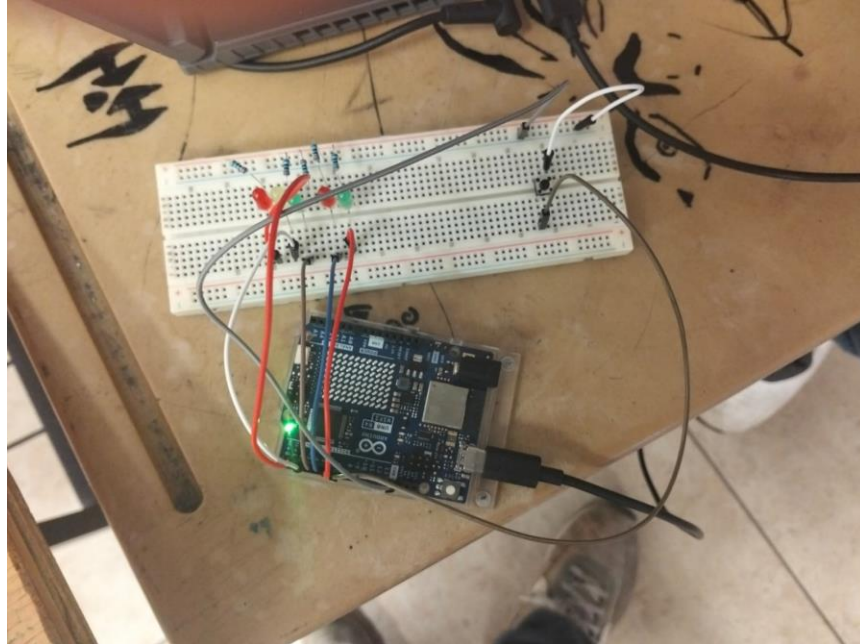
- Identificar el comportamiento bloqueante de la función `delay()` en Arduino.
- Comprender por qué `delay()` impide la ejecución concurrente de múltiples tareas dentro de `loop()`.
- Observar la diferencia entre el comportamiento esperado (LEDs parpadeando de forma independiente) y el comportamiento real del programa.
- Sentar las bases para comprender por qué se recomienda el uso de `millis()` como alternativa no bloqueante.

3. Material utilizado

- Arduino UNO R4 WiFi
- Protoboard
- 3 LEDs de colores distintos
- 3 resistencias limitadoras de corriente
- Cables de conexión (jumpers)
- Cable USB para alimentación y programación

4. Diagrama de conexión

Los LEDs se conectaron a los pines digitales 2, 3 y 4 del Arduino, cada uno en serie con su respectiva resistencia limitadora hacia GND. A continuación se muestra el montaje físico utilizado durante la práctica:



5. Análisis del código: el antipatrón

El programa define tres pines de salida (LED1, LED2 y LED3) y, dentro de loop(), controla cada LED de forma secuencial:

```
digitalWrite(LED1, HIGH); delay(500); digitalWrite(LED1, LOW); delay(500);  
digitalWrite(LED2, HIGH); delay(1000); digitalWrite(LED2, LOW); delay(1000);  
digitalWrite(LED3, HIGH); delay(1500); digitalWrite(LED3, LOW); delay(1500);
```

El problema central es que la función delay() detiene por completo la ejecución del microcontrolador durante el tiempo indicado. Mientras el Arduino está "esperando" dentro de un delay(), no puede atender ninguna otra instrucción: ni actualizar otro LED, ni leer un botón, ni responder a una petición de red.

Esto significa que, aunque el código parece describir tres parpadeos independientes y simultáneos, en realidad el microcontrolador los ejecuta uno después del otro, de forma estrictamente secuencial. El LED2 no puede empezar su ciclo hasta que el LED1 termine por completo el suyo (encendido y apagado), y lo mismo ocurre entre LED2 y LED3.

6. Conclusiones

Esta práctica evidenció de forma clara por qué delay() es considerado un antipatrón cuando se necesita manejar múltiples tareas con temporizaciones distintas dentro de un mismo programa. Aunque el código es sencillo de escribir y de entender, su comportamiento real no corresponde a la intención original de tres parpadeos independientes, sino a una secuencia de bloqueos consecutivos que se van sumando.

Este ejercicio sirve como punto de partida para introducir el uso de millis(), una alternativa no bloqueante que permite controlar múltiples tiempos de forma verdaderamente independiente, sin detener la ejecución del resto del programa.